

In this exercise, you'll work with Kiro to identify and fix security vulnerabilities in your chatbot application. Security is the top priority for any production application, and this exercise will help you implement industry best practices.

What You'll Learn

By the end of this exercise, you will have:

- Identified common security vulnerabilities in web applications
- Implemented security best practices for AWS serverless applications
- Hardened your chatbot against common attack vectors
- Learned to work iteratively with Kiro on security improvements
- Applied the principle of least privilege to your AWS resources

Step 1: Security Assessment and Implementation

Ask Kiro to perform a comprehensive security review and implement fixes for your chatbot application.

Please perform a comprehensive security assessment of my chatbot application and implement fixes for any security concerns you find.

Focus on these areas:

- API Gateway security configurations
- Lambda function permissions and environment variable encryption
- S3 bucket policies and access controls
- CloudFront security headers
- Input validation and sanitization
- Authentication and authorization gaps
- Network security and CORS configurations

For each security fix you implement, please explain:

1. What security risk it addresses
2. How the fix works
3. Any impact on functionality

Test each fix to ensure the application still works correctly after the security improvements.

🔒 Security-First Approach

As a product manager working with your development team (Kiro), your role is to ensure security requirements are met while maintaining functionality. Be prepared for an iterative process of testing and refinement.

Expected Security Improvements

Kiro should implement fixes for common issues in these areas:

API Gateway Security

Lambda Function Security

S3 and CloudFront Security

- Request throttling and rate limiting
- API key requirements or authentication
- Input validation at the gateway level
- Proper error handling without information disclosure
- CORS configuration restrictions
- Request/response size limits

Step 2: Validate Security Improvements

After Kiro implements the security fixes, thoroughly test the application to ensure both security and functionality.

Please help me validate the security improvements by:

1. Testing that the chatbot still functions correctly after security changes
2. Verifying that security headers are properly configured
3. Confirming that API rate limiting is working
4. Checking that error messages don't expose sensitive information
5. Validating that all AWS resources follow least privilege principles

Show me the results of these security validation tests.

Security Testing Checklist

Use this checklist to verify your security improvements:

- ☐ **Functionality Test:** Chatbot sends and receives messages correctly
- ☐ **Rate Limiting:** API requests are properly throttled
- ☐ **Error Handling:** Error messages don't reveal system details
- ☐ **Headers:** Security headers are present in HTTP responses
- ☐ **Permissions:** AWS resources have minimal required permissions
- ☐ **Input Validation:** Malicious input is properly sanitized
- ☐ **Logging:** Security events are logged appropriately

Step 3: Iterative Security Refinement

Security hardening is often an iterative process. Work with Kiro to refine and improve the security posture.

Let's do a final security review. Please:

1. Double-check all implemented security measures
2. Look for any remaining vulnerabilities we might have missed
3. Suggest additional security enhancements for production readiness
4. Verify that all security changes are properly documented

If you find any issues during testing, please fix them and retest.

⚠️ Iterative Process

Security hardening involves trial and error. If Kiro reports that security work is complete but testing reveals issues, provide specific feedback about what's not working and ask for fixes.

Expected Results

After completing this exercise, you should have:

- ☒ **Security Assessment:** Comprehensive review of all application components
- ☒ **Vulnerability Fixes:** All identified security issues addressed
- ☒ **Best Practices:** Industry-standard security measures implemented
- ☒ **Functional Testing:** Application works correctly with security improvements
- ☒ **Documentation:** Security measures and configurations documented
- ☒ **Production Readiness:** Application hardened for production deployment

Common Security Issues and Fixes

API Gateway

Lambda Function

Frontend Security

Authentication

Security Issues & Solutions

Common Issues:

- No rate limiting or throttling
- Verbose error messages exposing system information
- Missing input validation
- Inadequate CORS configuration

Recommended Fixes:

- ☒ Implement API throttling and usage plans
- ☒ Configure custom error responses
- ☒ Add request validation models
- ☒ Restrict CORS to specific domains

Impact: Prevents API abuse, reduces information disclosure, and ensures proper access control.

Troubleshooting Security Implementation

- ▶ Application Breaks After Security Changes
- ▶ Security Headers Not Applied
- ▶ Rate Limiting Too Restrictive

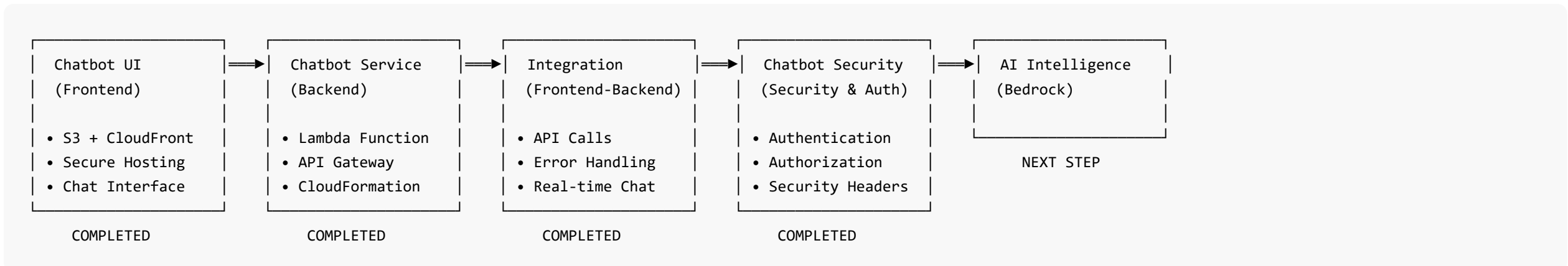
Security Architecture Overview

- ▶ View Secured Application Architecture

Next Steps

Congratulations! Your chatbot application is now security-hardened and ready for the final enhancement. With robust security measures in place, it's time to add AI intelligence to make your chatbot truly smart and engaging.

Workshop Progress



[Let's Add AI Intelligence With Bedrock](#)